

Chp 7:- BACKTRACKING

Backtracking is another algorithmic design technique used to improve the efficiency of exhaustive search.

Backtracking method can be applied to both optimization & as well as non-optimization problems.

Generally, backtracking method is applied to the combinatorial problems which cannot be solved in a polynomial time.

For ex, n -queen's problem, Hamiltonian circuit problem, sub-set-sum problem, etc.

MADHUSUDHAN M.V.

Dept. of CSE

Backtracking is an algorithmic design technique used to solve problems with large search space by systematically trying some possibilities (towards solution) & eliminating some possibilities (which doesn't promise to give solution).

Backtracking method is used to solve the problems for which, a sequence of objects is to be selected from a set such that sequence satisfies the problem constraint.

The principal idea of backtracking is to construct solutions by considering one component at a time & evaluating such partially constructed solutions in 2 stages

- 1) If the partially constructed solution still leads to a complete solution without violating the problem constraints, then we will take the next legitimate input among the remaining.
- 2) If the partially constructed solution violates the problem constraint $\&$ it doesn't lead to the complete solution, then algorithm backtracks to the previous step & look for alternative choice with next option.

Most of the backtracking problems are solved by using State-Space tree. The different choices we made during backtracking are represented as nodes of the state space tree.

In a state-space tree

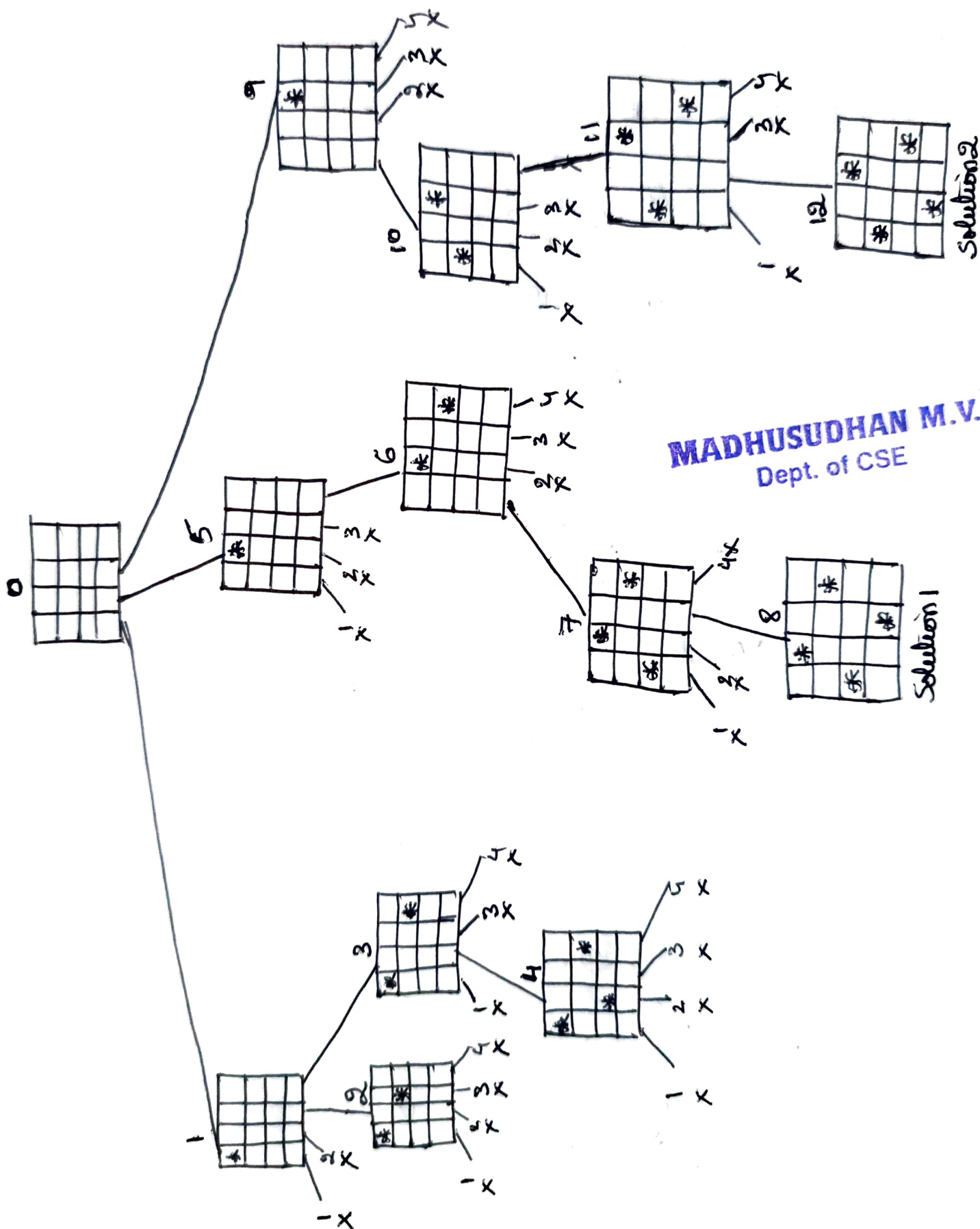
MADHUSUDHAN M.V.
Dept. of CSE

- * Root represents an initial state
- * Internal node represents promising partial solution.
- * External $\&$ leaf node represents either a non-promising $\&$ partial solution $\&$ final solution.

7.2 N-Queen's problem

In this problem, we need to place n queens on an $n \times n$ size chessboard so that, no 2 queens attack each other by being on the same row $\&$ same column $\&$ same diagonal.

For $n=1$, only one trivial solution. For $n=2$ & $n=3$, there is no solution. So, let us consider 4-queen problem.



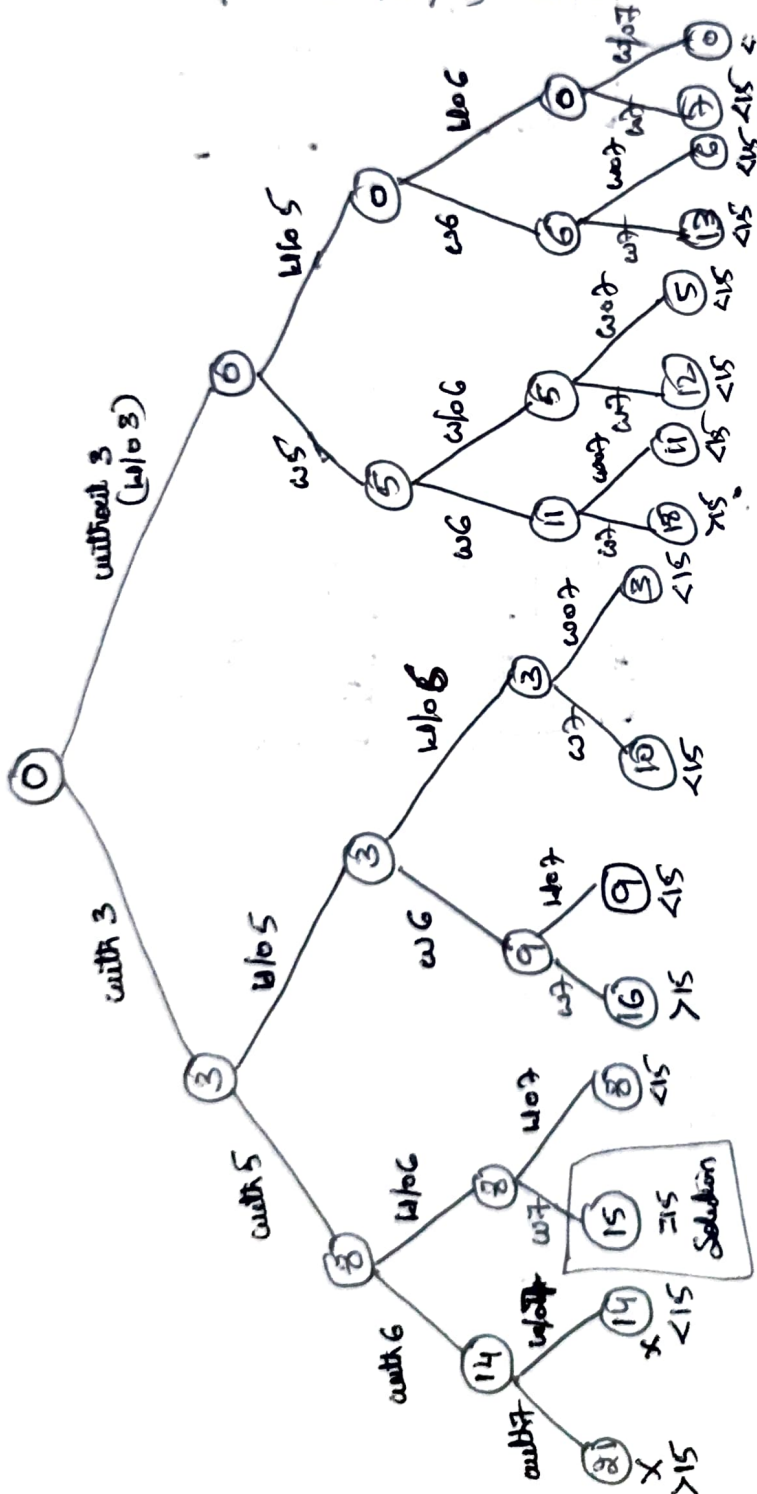
MADHUSUDHAN M.V.
Dept. of CSE

7.3 Sum of Subsets problem

Let S be the given set of n positive integers, i.e., $S = \{s_1, s_2, \dots, s_n\}$. We need to find the subset of given set in such a way that whose sum is equal to given positive integer d .

For this problem, it is better to sort the elements of set S .

~~Let $S = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ & $d = 2$~~ Let $S = \{3, 5, 6, 7\}$ & $d = 15$.



Subnd is 3,5,7

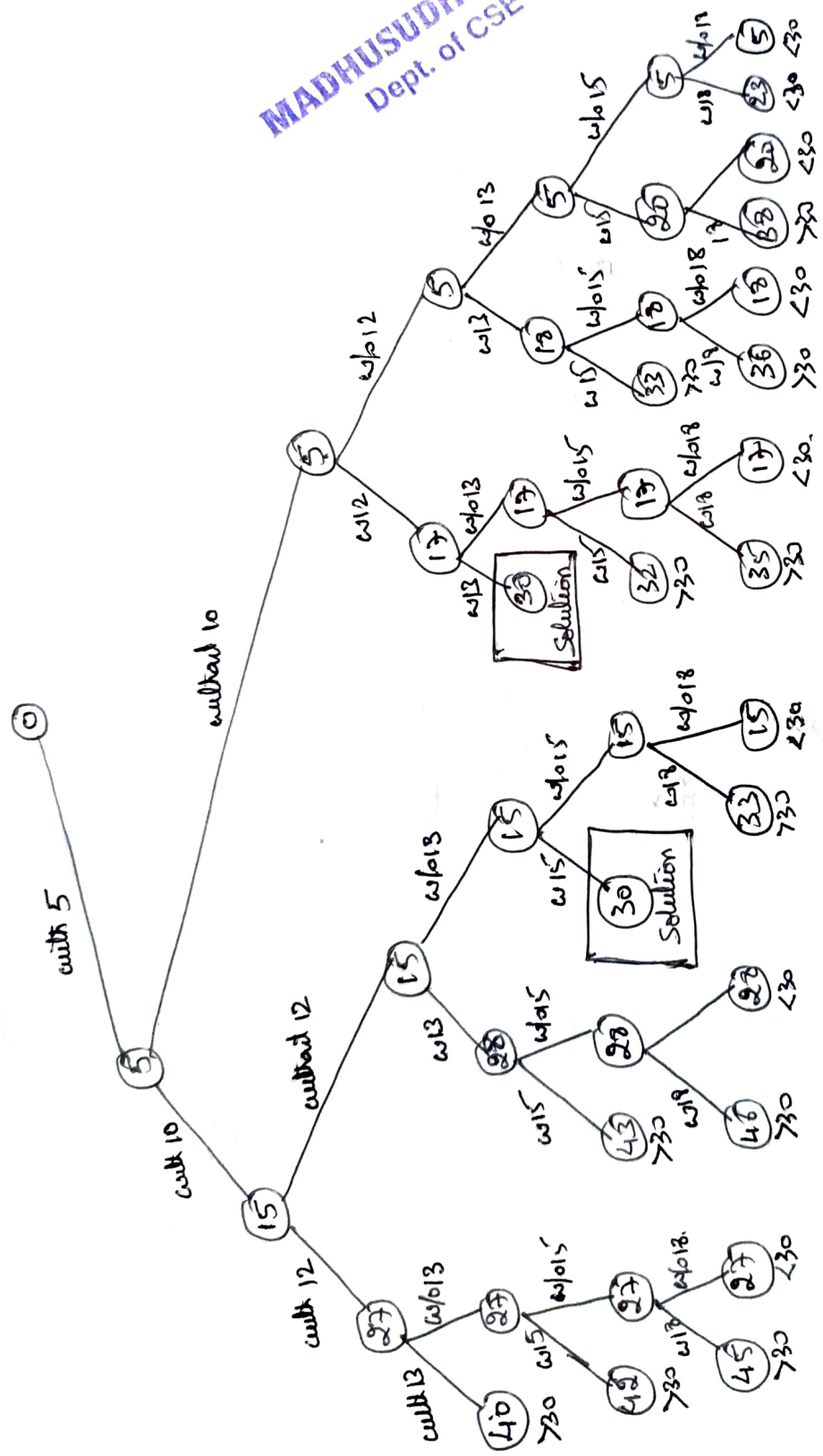
MADHUSUDHAN M.V.
Dept. of CSE

Problem 1

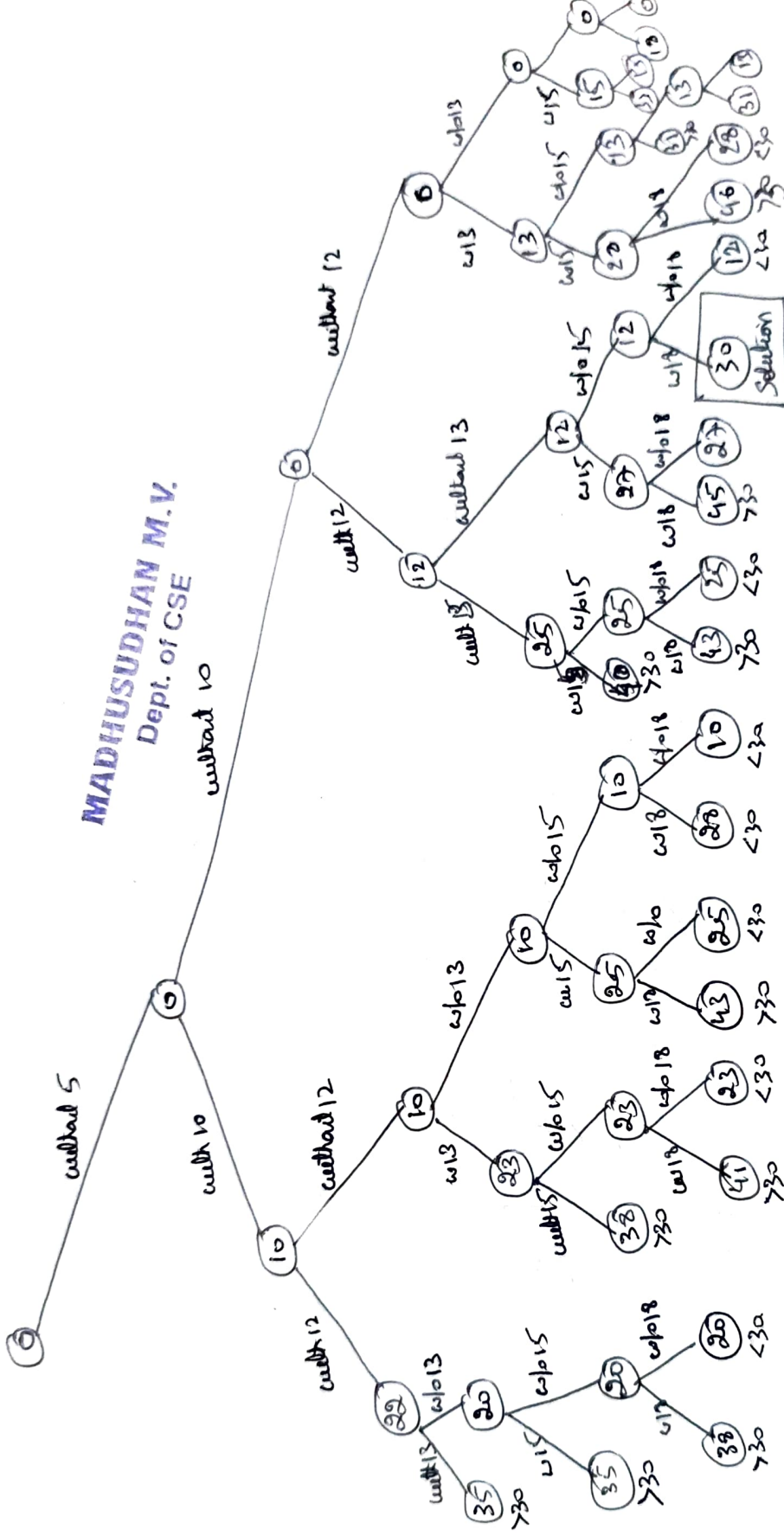
Apply backtracking method to solve subset sum problem for the instance $n=6, d=30, S=\{5, 10, 12, 13, 15, 18\}$

Solution

MADHUSUDHAN M.V.
Dept. of CSE



Right subtree is ~~in~~ continued in next page, (PTO)



There are 3 optimal solutions for this problem. They are: ① {5, 10, 15}, ② {5, 12, 13}, ③ {12, 13}

7.4

Hamiltonian Circuit Problem

4

In this problem, we need to find whether Hamiltonian Circuit is there or not in the given graph.

"Hamiltonian Circuit can be defined as a path that visits all the graph's vertices exactly once before returning to the starting vertex."

For the given graph, we can start from any vertex. Let us start from vertex 'a' of graph shown in fig 7.1. The state-space tree to find the Hamiltonian Circuit for the given graph is as shown in fig 7.2.

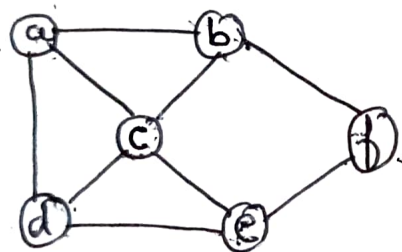


Fig 7.1

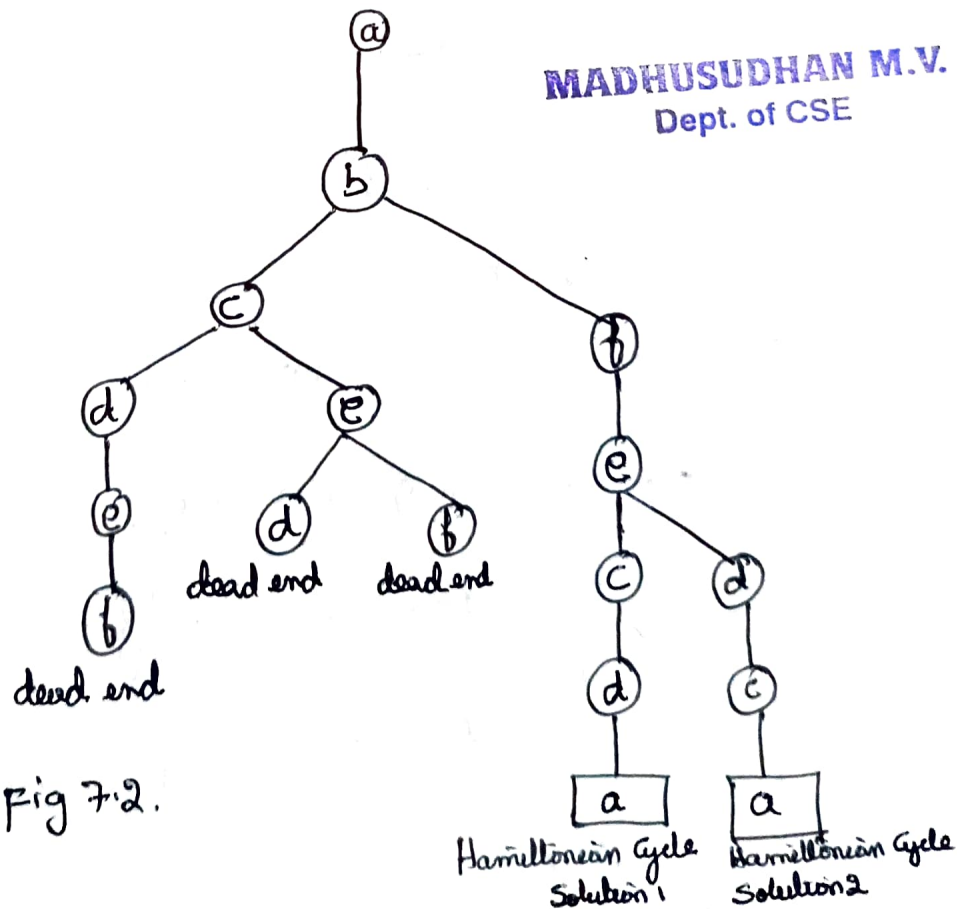


Fig 7.2.

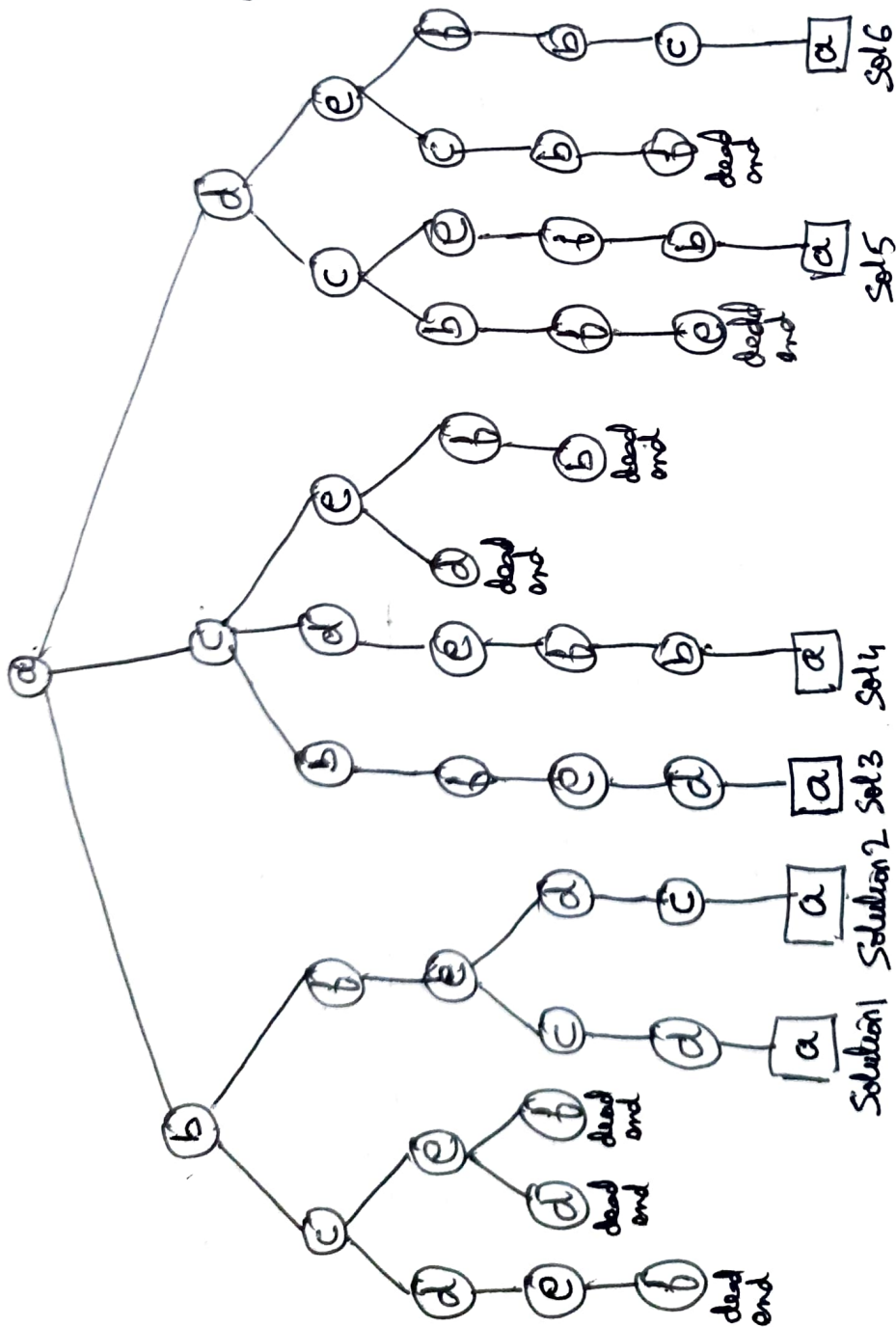
NOTE:-

From (a), we can move to either (b) or (c) or (d). In order to break the tie, we will use alphabetical order. So, we will move to (b).

Whenever, it is not possible to move ~~on~~ from any vertex to any other vertex, it indicates dead end is reached.

~~Def~~ ^{cc} Dead end node is a node, from which it is not possible to move to any other node."

Fig 7.3 shows all possible solutions



MADHUSUDHAN M.V.
Dept. of CSE

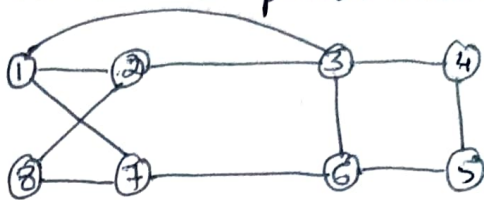
Fig 7.3. Complete state-space tree

Hamiltonian cycles are,

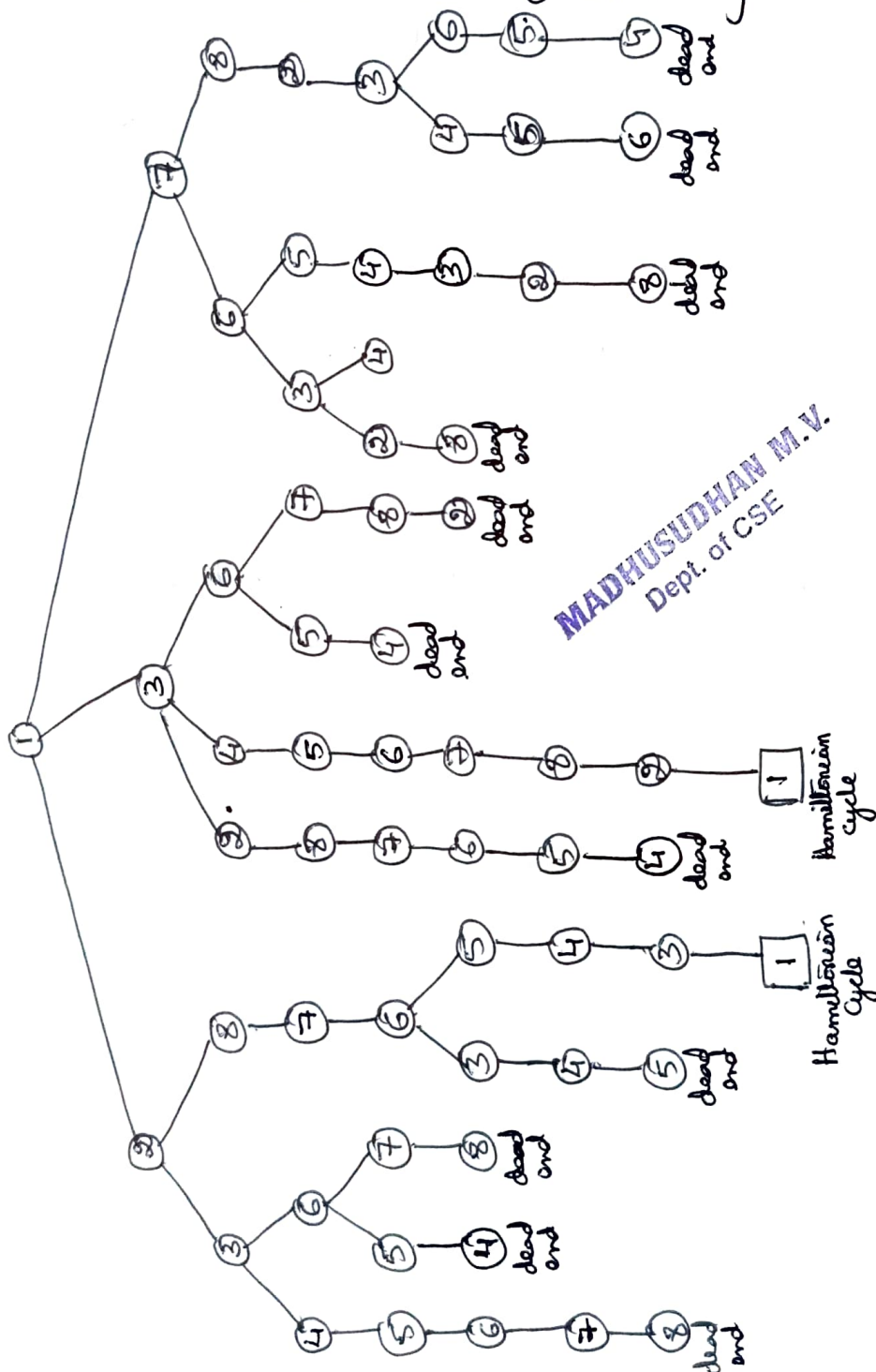
- 1) a-b-b-e-c-d-a
- 2) a-b-b-e-d-c-a
- 3) a-c-b-b-e-d-a
- 4) a-c-d-e-f-b-a
- 5) a-d-c-e-f-b-a
- 6) a-d-e-f-b-c-a

Problem 2

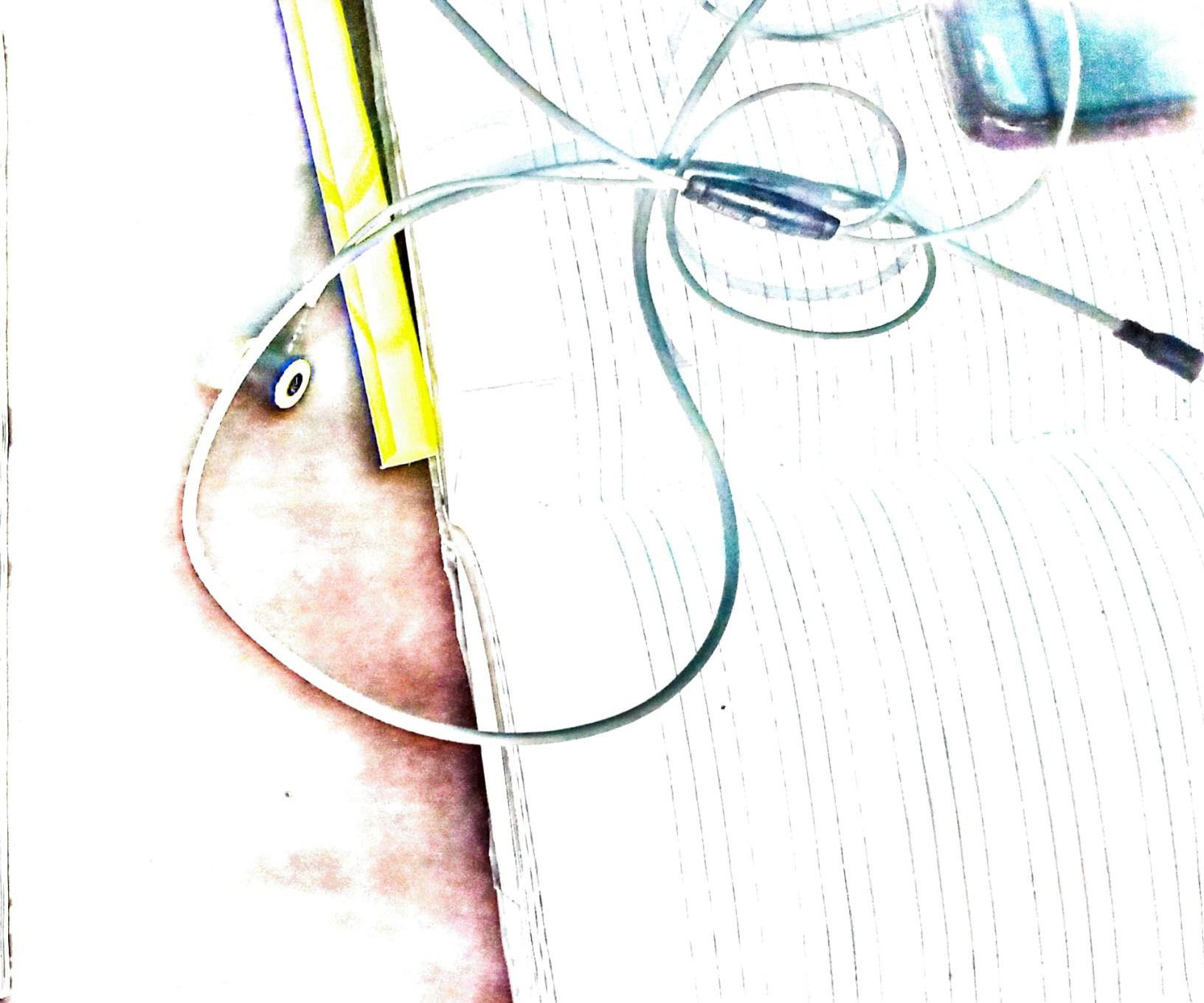
Apply backtracking method to obtain all possible Hamiltonian cycles for the given graph.



Solution:- Let us take node ① as starting node.



- For the given graph, there are 2 Hamiltonian cycles,
- 1) 1-2-8-7-6-5-4-3-1
 - 2) 1-3-4-5-6-7-8-2-1



Chp 8BRANCH & BOUND TECHNIQUE

Branch & bound is another algorithmic design technique used to get optimal solution for various optimization problems especially, discrete & combinatorial problems.

In order to get optimal solution, greedy method & the dynamic programming methods are used. But, in some cases, these 2 methods fail. When these 2 methods fail, then branch & bound is another option to get optimal solution.

Branch & bound technique uses BFS method & it uses queue as data structure.

Disadvantages of backtracking

In backtracking, every node of the state-space tree is expanded even though it doesn't give solution.

After getting one solution also, we will continue to get another solution.

Consumes more space & time. Hence, it is not time efficient & also ^{not} space efficient.

These disadvantages can be overcome by branch & bound technique.

MADHUSUDHAN M.V.
Dept. of CSE

3.1) General method

In branch & bound, we are not going to expand all the nodes of the state space tree. Instead of that,

Carefully selected criterion determines which node to expand & when.

Whenever we found the solution, then we will stop there itself instead of going to find next possible solution.

When compared to backtracking, branch - & bound requires 2 additional concepts

1) for every node of the state space tree, we need to find lower & upper bound by adding further components to the partially constructed solution represented by the node.

2) value of best solution seen so far

MADHUSUDHAN M.V.
Dept. of CSE

After finding the node's bound value, then it is compared with the best solution seen so far

* if the bound value is not better than the best solution so far, then that node is considered as a non-promising node, then we will terminate there itself (i.e., no branching).

* if the bound value is better than the best solution, then we will consider that bound value as a best solution & from there onwards we will branch the tree.

8.2 Job assignment problem

In this problem, there are n number of peoples & n number of jobs. These n jobs should be assigned to n peoples in such a way that, one job should be assigned only one person (i.e. one person is assigned to one job).

Objective of this problem is to find job assignment with minimum total cost (i.e. all jobs should get completed within minimum number of days).

Problem 1

job1	job2	job3	job4	
9	2	7	8	Person a
6	4	3	7	Person b
5	8	1	8	Person c
7	6	9	4	Person d

MADHUSUDHAN M.V.
Dept. of CSE

Solution:

Now, the question is, how to get the lower bound for the initial node (root) without solving the problem.

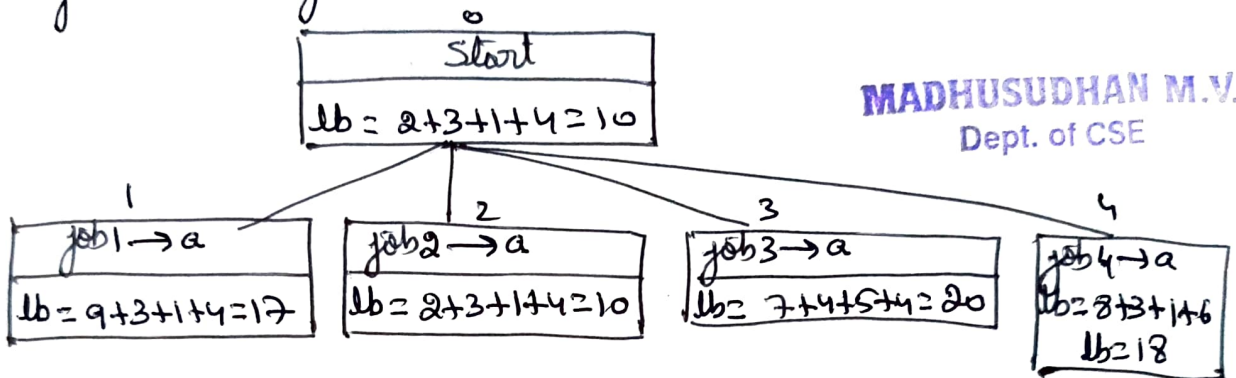
The best simple method is, select the minimum element from each of the row. For this example, this sum is $2+3+1+4=10$. But, it is not the legitimate solution because it violates the problem constraints. (3 & 1 taken from the same column). But, still we can start from this by constructing partial solutions & finally we will get the legitimate optimal solution.

start
$lb = 2+3+1+4 = 10$

From now onwards, we will look into partially constructed solutions by considering one component at a time with all the options/choices/possibilities.

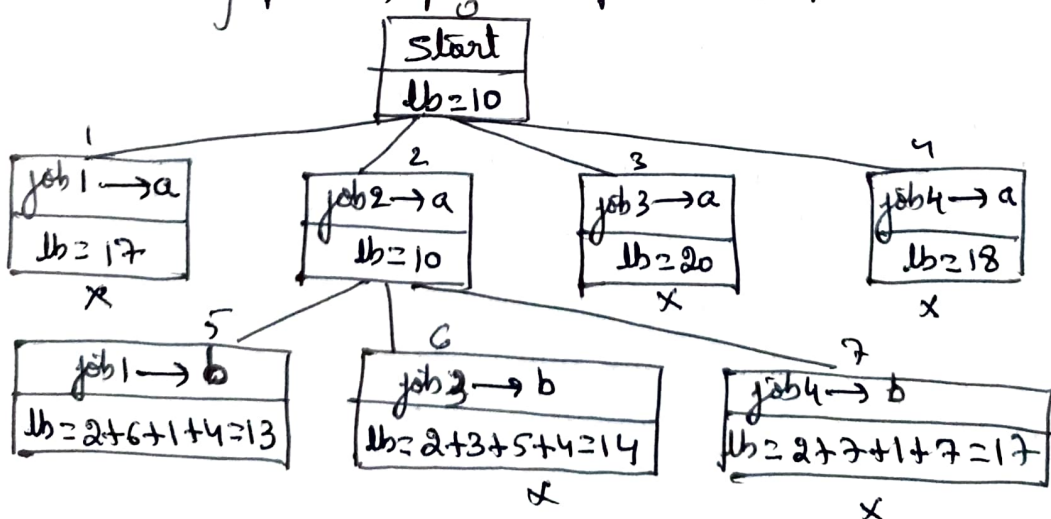
Now, we will consider only person a & we will find out what is the lower bound if we assign job1 to a, job2 to a, job3 to a @ job4 to a.

MADHUSUDHAN M.V.
Dept. of CSE



Among these 4 nodes (node1 to node4), node 2 is more promising node as its bound value is minimum. So, we will branch only node 2. It indicates, job2 is fixed to person a. From now onwards, we should not consider any value from 2nd column.

Now, we need to assign remaining jobs, job1, job3 & job4 to remaining persons, person b, person c & person d.



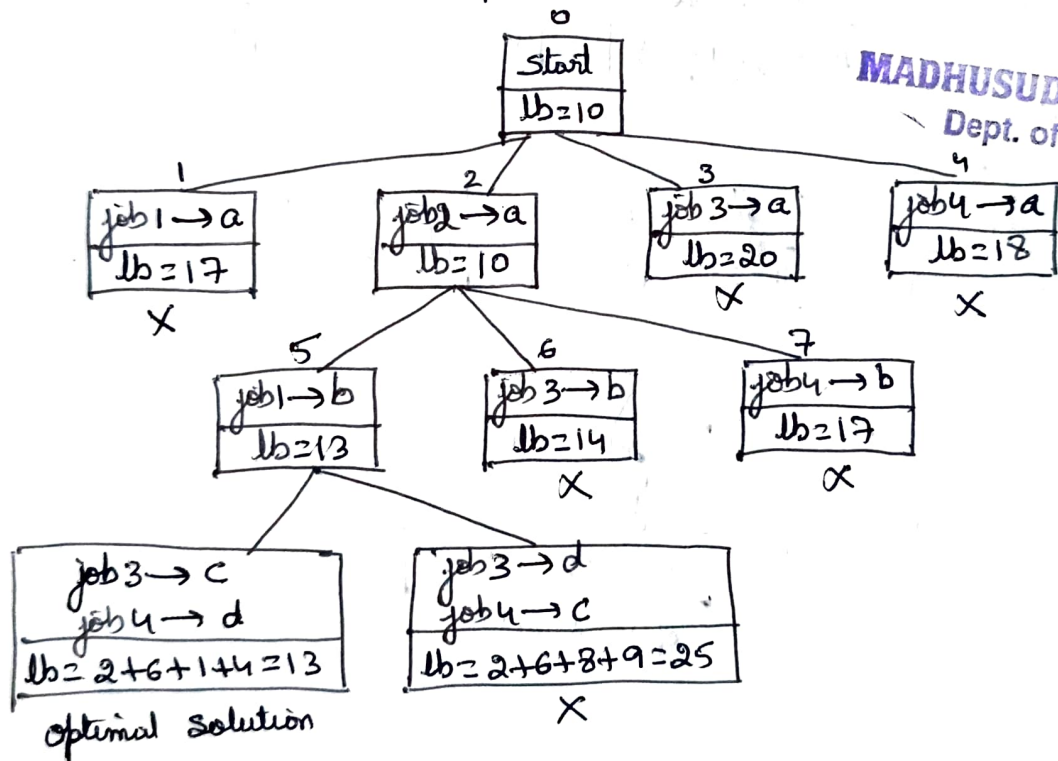
(9)

Now, we will choose node 5 as its bound value is minimum.
 Now, it is concluded that, job 2 is assigned to person a & job 1 is assigned to person b.

Now, we have to assign remaining jobs job 3 & job 4 to remaining persons person c & person d.

So, we left with only 2 options. If we assign job 3 to person c then obviously, we have to assign job 4 to person d. Another possibility is, if assigning job 3 to person d & job 4 to person c.

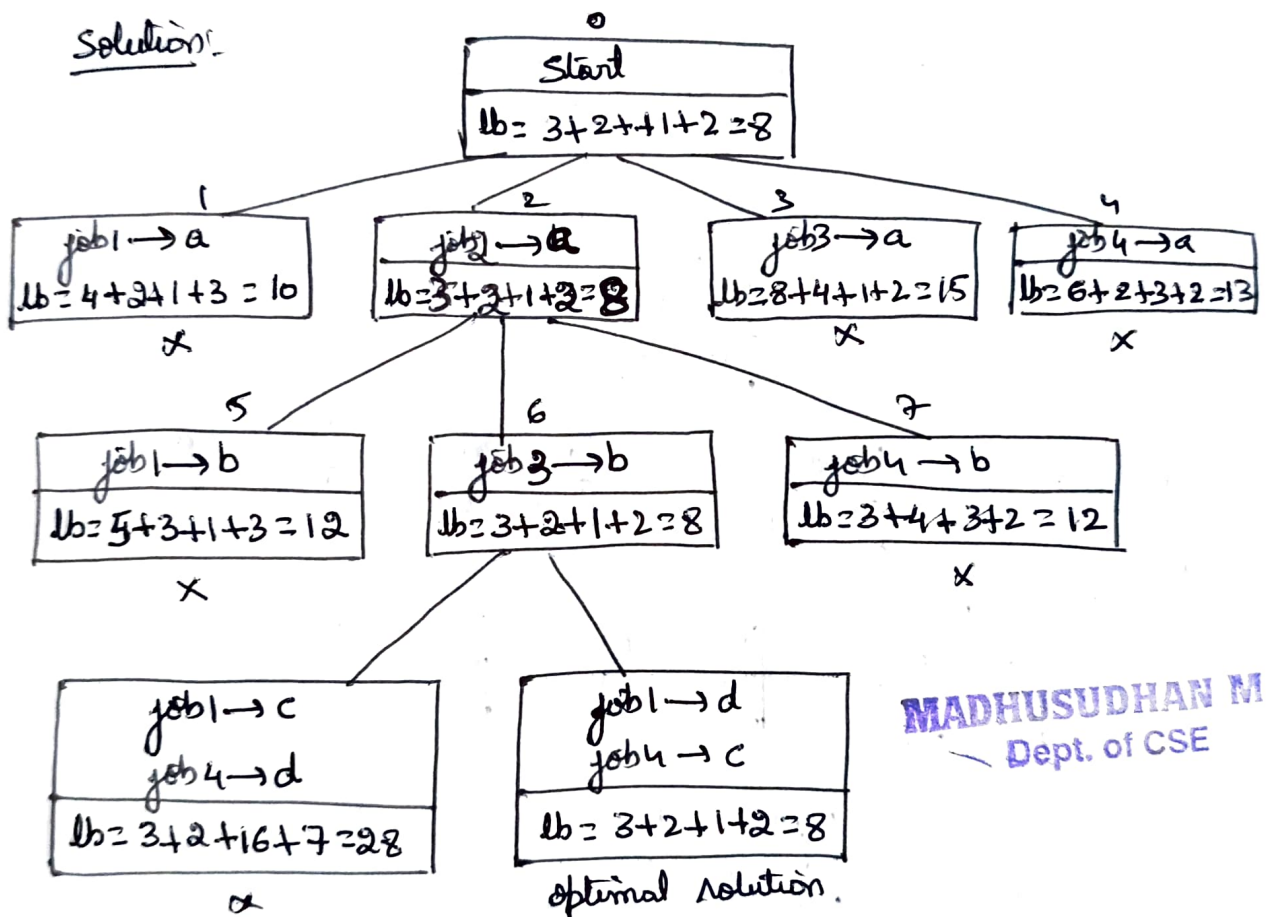
Thus, we have Complete state-space tree as shown below



Problem 2 Write the state-space tree for the given matrix

	job1	job2	job3	job4
person a	4	3	8	6
Person b	5	7	2	4
Person c	16	9	3	1
Person d	2	5	3	7

Solution:-



MADHUSUDHAN M.V.
Dept. of CSE

job1 → d
job2 → a
job3 → b
job4 → c

} gives us the optimal solution of 8.

8.3 Knapsack Problem

Problem statement :- Given n items of known weights w_i & having values v_i & knapsack capacity W . We need to find the most valuable subset of the items that fit in the knapsack.

For our convenience, we are going to order the given n items in decreasing order based on the values per unit weight i.e.,

$$v_1/w_1 \geq v_2/w_2 \geq v_3/w_3 \geq \dots \geq v_n/w_n$$

To solve this problem using branch & bound technique, we will construct state-space tree in the form of binary tree.

Left child of a node indicates inclusion of next item ($i+1^{\text{th}}$ item) & right child of a node indicates exclusion of next item.

In each node, we will store total weight ' w ' of the selected items, total value V of the selected items & upper bound value, ub .

For each node, upper bound value, ub , is calculated using the formula

$$ub = V + (W - w) (V_{i+1} / w_{i+1})$$

$V \rightarrow$ value of i^{th} item

$W \rightarrow$ maximum knapsack item

$w \rightarrow$ weight of i^{th} item

$V_{i+1} \rightarrow$ value of next i^{th} item

$w_{i+1} \rightarrow$ weight of next i^{th} item

MADHUSUDHAN M.V.
Dept. of CSE

Problem ③

Item	Weight	Value
1	4	40
2	7	42
3	5	25
4	3	12

For the given instance, obtain the optimal solution for knapsack problem by applying branch & bound method.

Assume knapsack capacity is $W=10$.

MADHUSUDHAN M.V.
Dept. of CSE

Solution:

Item	Weight	Value	Value/weight
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

Find the value per unit weight of each item & sort them in decreasing order.

But for this example, it is in decreasing order itself.

Root of the state-space tree is a node with no items are selected (initially) & hence no value. So, upper bound is

$$ub = 0 + (10-0)(40/4) = 100$$

Node 1: with item 1, $w=4$, $v=40$ & $ub = 40 + (10-4)(42/7) = 76$

Node 2: without item 1, $w=0$ itself & $v=0$ only $ub = 0 + (10-0)(42/7) = 60$

When we compare upper bounds of node 1 & node 2, node 1 upper bound is maximum. So, we will branch from node 1.

Node 3: with item 2, total weight becomes $w=4+7=11 > W$. Hence, not feasible solution

Node 4: without item 2, $w=4$ & $v=40$ itself, $ub = 40 + (10-4)(25/5) = 70$

Node 5: with item 3, total weight becomes $w=4+5=9$ & $v=40+25=65$

$$ub = 65 + (10-9)(12/3) = 69$$

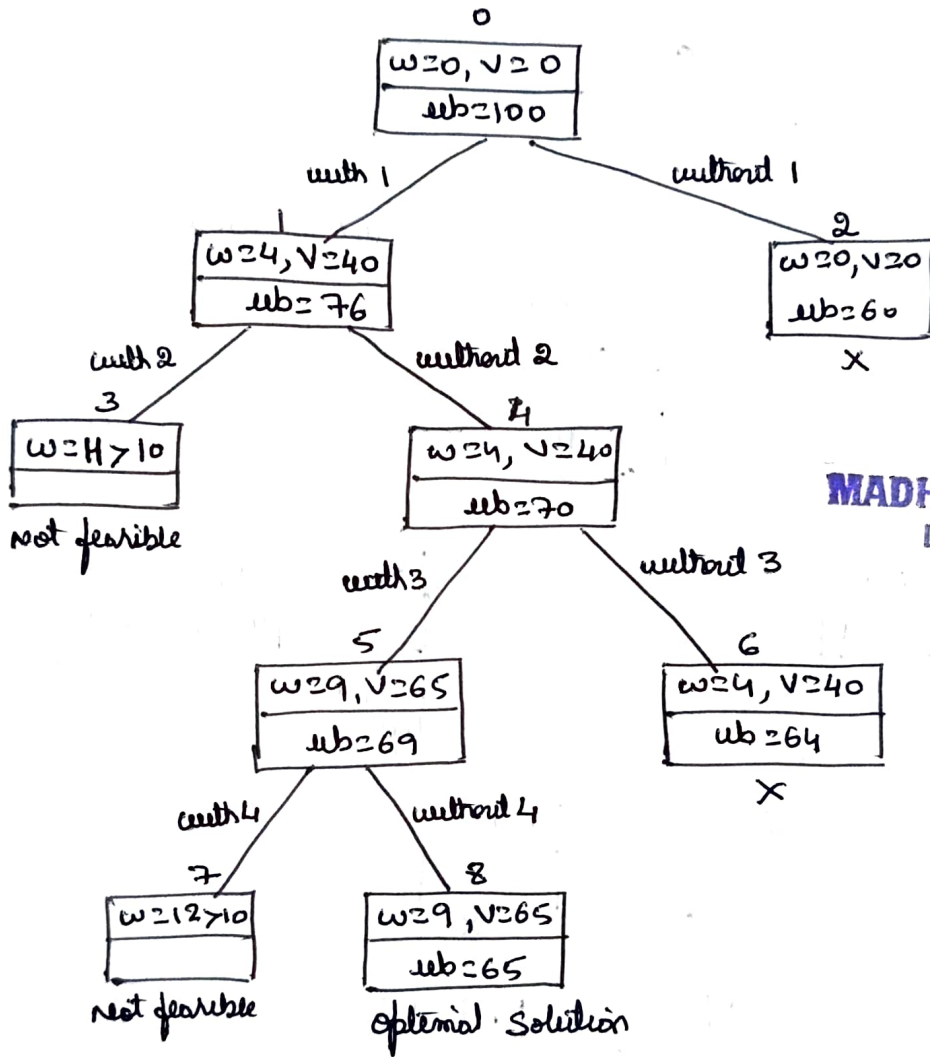
Node 6:- without item 3, $w=4$ & $v=40$ itself

$$ub = 40 + (10-4)(12/3) = 64$$

Node 7:- with item 4, total weight $w = 9+3 = 12 > 10$. not feasible solution.

Node 8:- without item 4, total weight $w = 9$ & $v=65$ itself

$$ub = 65 + (10-9)(0) = 65$$

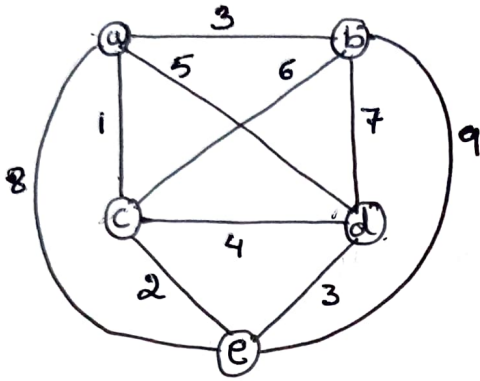


MADHUSUDHAN M.V.
Dept. of CSE

NOTE:- whatever the example, right child of root (node 2) will never continue, it stops there only because, it is constructed without considering 1st item but 1st item itself is having maximum profit per unit weight $\therefore$ we have sorted like that only

8.4 Travelling Salesman Problem

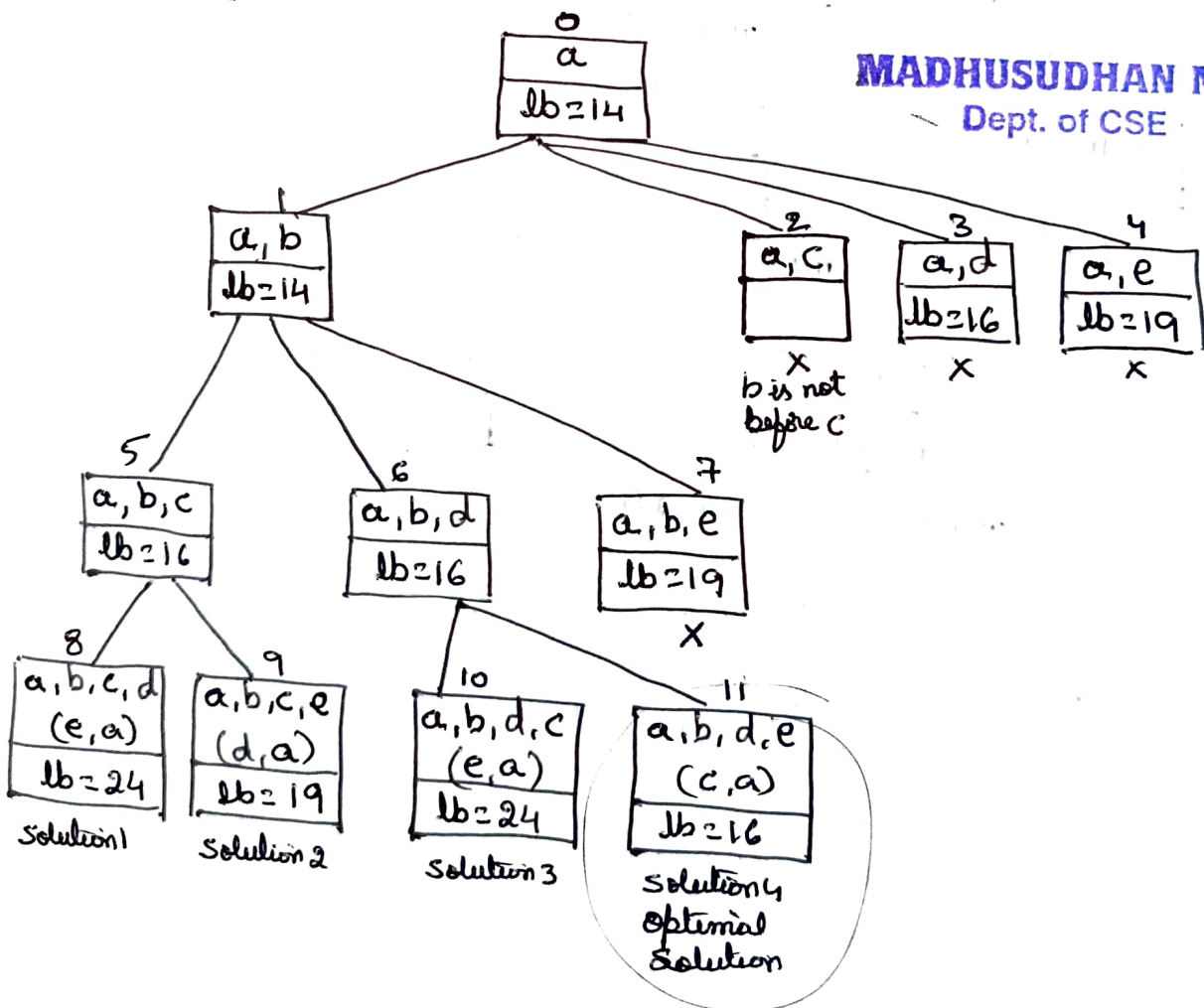
In this problem, we need to find minimum cost tour & that tour should cover all the cities exactly once before returning to the starting city.



In order to find lower bound for each node, formula is $lb = \lceil S/2 \rceil$

For every node, we will consider 2 nearest cities & we will get the sum of 2 nearest cities of every node. This is S.

MADHUSUDHAN M.V.
Dept. of CSE



Node 0:- Let us take @ as starting vertex. Lower bound is calculated by considering 2 minimum edges from each of the vertex & summing them & finally dividing by 2.

$$lb = \frac{(1+3) + (3+6) + (1+2) + (3+4) + (2+3)}{2} = 14$$

$\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$
 2 min^m edges of @ 2 min^m edges of @ 2 min^m edges of @ 2 min^m edges of @ 2 min^m edges of @

MADHUSUDHAN M.V.
Dept. of CSE

Now, from @ we are having 4 options, @ to B, @ to C, @ to D, @ to E. So, we have 4 branches from node 0.

Node 1:- Here, we are moving from @ to B. So, this edge should be considered while considering 2 minimum edges while calculating lb.

$$lb = \frac{(1+3) + (3+6) + (1+2) + (3+4) + (2+3)}{2} = 14$$

Node 2:- when we visit all the nodes & while returning to starting vertex, then that edge should be minimum. In this example, it is from @ to @. So, we should not move from @ to @.

Node 3:- Here, we are moving from @ to D. So, this edge should be considered while considering 2 minimum edges of @ & D.

$$lb = \frac{(1+5) + (3+6) + (1+2) + (5+3) + (2+3)}{2} = 16$$

Node 4 Similarly for node 4; $lb = \frac{(1+3) + (3+6) + (1+2) + (3+4) + (2+3)}{2} = 14$.

Among node 1, 2, 3 & 4, promising node is node 1. So, we will branch from node 1. That means, from @, we visited B. Now from B, there are 3 options, B to C, or B to D or B to E.

Node 5:- edges ab & bc should be considered.

$$\text{So, } lb = \frac{(1+3) + (3+6) + (6+1) + (3+4) + (2+3)}{2} = 16$$

Node 6:- Edges ab & bd should be considered. So,

$$lb = ((1+3) + (3+7) + (1+2) + (7+3) + (2+3)) / 2 = 16$$

Node 7:- Edges ab & be should be considered. So,

$$lb = ((1+3) + (3+9) + (1+2) + (3+4) + (2+9)) / 2 = 19.$$

Among node 5, 6 & 7, node 5 & node 6 both are having same lower bound. So, we will branch from both the nodes.

Node 8:- From here onwards, we will find actual cost of the selected path. We won't calculate lb, we will calculate directly the cost of path.

In node 5, it is clear that, we started the tour from @ & we reached ©. So, from ©, there are 2 options, either we can reach d or e. If we reach from © to d, then remaining option is from d to © & © to @. That means, path is $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow a$ & its cost is 24.

Similarly node 9, 10 & 11

MADHUSUDHAN M.V.
Dept. of CSE

Node 9:- path is $a \rightarrow b \rightarrow c \rightarrow e \rightarrow d \rightarrow a$ & its cost is 19

Node 10:- path is $a \rightarrow b \rightarrow d \rightarrow c \rightarrow e \rightarrow a$ & its cost is 24

Node 11:- path is $a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a$ & its cost is 16.

Chp 9: NP-Complete & NP-Hard Problems

Till now we discussed different types of algorithmic design techniques to solve the different types of problems.

We also did the analysis of every algorithm just to know the power of an algorithm in terms of its efficiency.

Each & every algorithm is having its own power. Power means efficiency of the algorithm. That means, in which situation this particular algorithm is most efficient, for which type of inputs, this algorithm works well.

Each & every algorithm has its own limitations in terms of its power. In this chapter, we are discussing about limitations of algorithms in terms of their power.

Most of the problems can be solved through algorithms. Among them, some of the problems can be solved within polynomial time & some of them can be solved but not within polynomial time.

polynomial terms means exponential terms. Ex: x^2, x^3, x^a, x^n, a^n , etc.

MADHUSUDHAN M.V.

Dept. of CSE

If the order of growth of any algorithm is in terms of exponential, then it is very time consuming & then we can say that, power of that algorithm is not within the polynomial time.

9.1 Lower-Bound arguments

In order to find the power/efficiency of an algorithm, we need to find their lower bounds.

Finding lower bound means, "estimating minimum amount of work to be done to solve the problem".

For example, for the following problems, we can obtain lower bound easily;

- * number of comparisons needed to sort an array of size n .
- * number of comparisons needed to search a element
- * number of additions needed to add 2 $n \times n$ matrices.

We can obtain the ^(lower bound) efficiency of any algorithm, just know, whether it can be improved or not. Steps to be followed are

- * obtain the asymptotic efficiency class for the given problem.
- * check, where this efficiency class fits in the hierarchy of efficiency class. That means, we need to check whether efficiency class of this problem lies in linear, quadratic, logarithmic or exponential.

- * Compare efficiency class of this algorithm with the other algorithm's efficiency of same problem type.

Let us understand these concepts by taking an example of sorting problem.

Let us consider selection sort. Its efficiency is n^2 . Let us take another sorting algorithm, merge sort & whose efficiency is $n \log_2 n$.

MADHUSUDHAN M.V.

Dept. of CSE

If we compare the efficiency of selection sort & merge sort, it clearly indicates that, efficiency of selection sort can be improved.

Whenever, we are comparing, we need to compare 2 algorithms of same problem type.

MADHUSUDHAN M.V.
Dept. of CSE

Once we find the lower bound of any algorithm, then it is very useful to know, whether the efficiency of algorithm can be improved or not.

Let us consider merge sort, whose lower bound is $n \log_2 n$. We know that, fastest sorting algorithm is quick sort whose lower bound is $n \log_2 n$. When we compare these 2, we can say that, there is a tightness (tight lower bound). It indicates, no need to improve merge sort.

If we consider lower bound of an linear search is n & fastest searching algorithm (binary search) has an efficiency of $\log_2 n$. This implies that, linear-search can be further improved in order to make it more efficient.

In order to obtain the lower bound, there are 4 methods

- 1) Trivial lower bounds
- 2) Information-theoretic arguments
- 3) Adversary arguments
- 4) Problem reduction.

MADHUSUDHAN M.V.
Dept. of CSE

9.1.1 Trivial lower bounds

→28→

We know that, every program/algorithm receives the data, process the data & then produces the data in the form of output.

In order to process the input data, first algorithm has to read the input data.

This, trivial lower bound is the simplest method for obtaining the lower bound of an algorithm.

In this method, we will obtain the lower bound by counting number of input data processed (by reading) & the number of outputs it produces.

Any algorithm that has to read all input data, process it & produces all its outputs, such a count gives us the trivial lower bounds.

MADHUSUDHAN M.V.
Dept. of CSE

For example,

- 1) Any algorithm for generating all permutations of n distinct items must be in the order of growth $n!$ because the size of the output is $n!$
- 2) Trivial lower bound for computing product of 2 $n \times n$ matrices is $\Omega(n^2)$ because any such algorithm has to process $2n^2$ elements in the input matrices & generates n^2 elements as output.

$$\underbrace{\begin{bmatrix} - & - \\ \vdots & \vdots \\ - & - \end{bmatrix}}_{n \times n} \times \underbrace{\begin{bmatrix} - & - \\ \vdots & \vdots \\ - & - \end{bmatrix}}_{n \times n} = \underbrace{\begin{bmatrix} - & - \\ \vdots & \vdots \\ - & - \end{bmatrix}}_{n \times n}$$

Disadvantages of trivial lower bound method

* Less useful

* Cannot be applied for problems like binary search, in

which, we are not processing the all input data.

- * for problem like TSP, trivial lower bound is $\Omega(n^2)$ because there are $n(n-1)/2$ distances (input) & $n+1$ cities as output. But, this trivial lower bound is not useful because there is no another known algorithm to solve this problem. So, no chance to compare with other algorithm's lower bound.

9.1.2 Information - theoretic arguments

MADHUSUDHAN M.V.
Dept. of CSE

Using this method, we can obtain lower bound from the information produced by the algorithm.

This method is based on the amount of information produced by the algorithm.

For example, Consider a "game of guessing number": first of all think of some number from 1 to n . Find answer by asking series of questions. Answer can be either YES @ NO. Output of this is the binary representation of the guessed number. Each answer will give information about each bit.

→ is 1 st bit is zero? → NO = 1	} = number is 9.
→ is 2 nd bit is zero? → YES = 0	
→ is 3 rd bit is zero? → YES = 0	
→ is 4 th bit is zero? → NO = 1	

MADHUSUDHAN M.V.
Dept. of CSE

To obtain lower bound by this method, a mechanism called decision tree is used. This method of finding lower bound is useful for the algorithms that are based on comparison methods @ sorting @ searching.

9.1.3 Adversary arguments

In this method, lower bound is obtained by measuring the amount of work required to reduce the large set of inputs to an single input along the most time consuming path.

Example ① playing "guessing game" between you & your friend.

- you have to pick a date & your friend will try to guess the date by asking YES/NO questions
- your purpose is to force your friend to ask as many questions as possible.

* Question is "is it in summer", your answer should be NO

* Question is "is the 1st letter of the month's name is in the 1st half of alphabet", your answer should be YES.

The idea behind this method for obtaining the lower bound is by asking questions repeatedly as many number as possible.

→ you did not pick up a date in advance at all, but construct a date according to the questions

→ Requirement is that, finally constructed date should be consistent to all your answers to the questions.

→ Looks like cheating, but it is a good way to find lower bound.

Adversary argument is a game between adversary & the unknown algorithm. The adversary has the input & the algorithm asks questions to the adversary about the input. The adversary tries to make the algorithm work the hardest by adjusting the input (consistently).

Example ② guessing a number between 1 & n using YES/NO questions

→ is it larger than x ?

→ is it lesser than y (20)?, etc

9.2 P, NP and NP-Complete Problems

Problems that can be solved within a polynomial time are called tractable problems & the problems that cannot be solved in polynomial time are called intractable problems.

Problems like, computing the product, finding GCD, sorting, searching, checking acyclicity of a graph, finding minimum cost spanning tree, finding shortest path, etc can be solved within polynomial time.

Any problem for which answer is either YES @ NO is called decision problems.

Algorithms used for decision problems are called decision algorithms.

MADHUSUDHAN M.V.

Dept. of CSE

~~Class - B~~

"Class-P problems are the decision problems that can be solved within polynomial time by using deterministic algorithms."

Here, p stands for polynomial.

We know that, algorithm is an step-by-step procedure, where each step (instruction) is nothing but an operation.

If every operation of an algorithm is uniquely defined then that algorithm is called as deterministic algorithm.

NOTE: operations are uniquely defined means, operation gives the same outcome/result every time it get executed.

9.1.4 Problem reduction

Problem reduction is a method in which a difficult unsolvable problem A is to be reduced to another solvable problem B with known algorithm.

The same principle is used for obtaining lower bounds. So, once we reduce the problem A to an known problem B, then the lower bound of B can be used as lower bound of A.

Example:- Let 'A' is an algorithm for finding the minimum spanning tree & 'B' is an algorithm of element uniqueness problem.

Consider a set $X = \{x_1, x_2, x_3, \dots, x_n\}$ from which we need to find for uniqueness of the elements. Assume we don't know the solution.

Now, we will transform X into a set of n points in the Cartesian plane by simply adding 0 as the points y. Co-ordinate; $(x_1, 0), (x_2, 0), (x_3, 0), \dots, (x_n, 0)$.

Let 'T' be a minimum spanning tree found for these set of points. Once we found the minimum spanning tree, then if any 2 edges of the minimum spanning tree are same, then given set is not an unique element set.

Lower bound of minimum spanning tree ($n \log_2 n$) itself is the lower bound of element uniqueness problem.

In this way, lower bound of one instance of a problem can be obtained using the lower bound of another instance of a problem.

If every operation of an algorithm is not uniquely defined, then that algorithm is called non-deterministic algorithm.

NOTE:- If same operation gives different results (not same results) on every time it get executed, then that operations are said to be not uniquely defined.

"Class-NP problems are the decision problems that can be solved within a polynomial time by using non-deterministic algorithms."

MADHUSUDHAN M.V.

Dept. of CSE

Examples:- Travelling Salesperson Problem, Graph Coloring, Knapsack problem, Hamiltonian circuit, etc.

Non-deterministic algorithm works in 2 stages

1) Non-deterministic stage (guessing stage)

In this stage, non-deterministic algorithm generates an arbitrary string S , that can be thought of as a candidate solution to the given instance I .

2) Deterministic stage (verification stage)

In this stage, deterministic algorithm is used & it takes both S & I as input & produces the output YES if S represents a solution to instance I . It produces output NO, if S is not a solution to instance I .

Here, NP stands for Non-deterministic polynomial time.

In the first stage, non-deterministic algorithm generates the solution (string s) by "guessing" & that guessed solution is "verified" by the deterministic algorithm in the II stage.

Whatever the class-P problems solved by deterministic algorithms that can also be solved by non-deterministic algorithm. Thus, we can write

$$P \subseteq NP$$

This relation is true because, if a problem is in P , we can use the deterministic algorithm that solves it in the verification stage of a non-deterministic algorithm.

Further, NP-class is divided into 2 categories:

- 1) NP-Complete
- 2) NP-Hard

MADHUSUDHAN M.V.
Dept. of CSE

So, complete classification of problems are as shown in fig 9.1

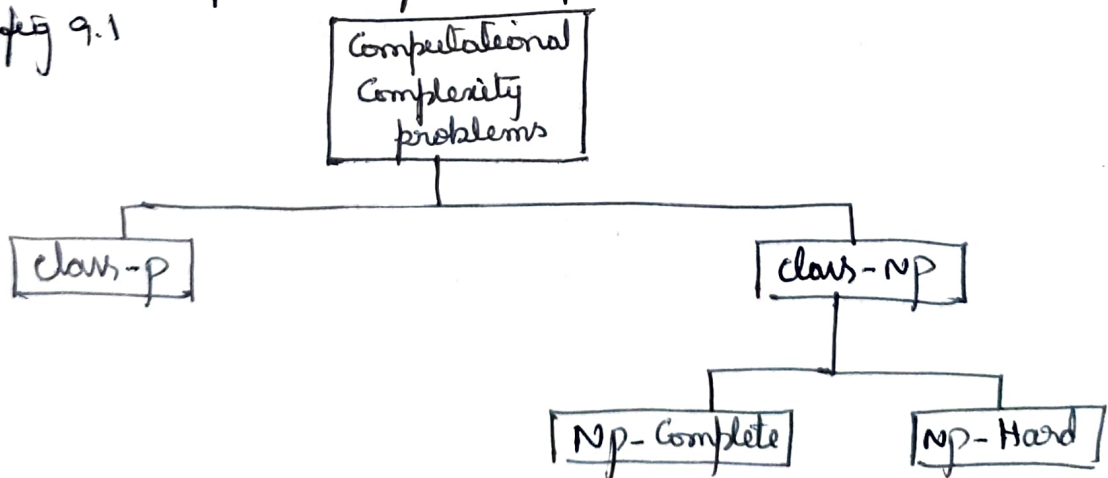


Fig 9.1

Equation $P \subseteq NP$ raises so many questions

- 1) $P \subseteq NP$ (P is a subset of NP) means some of the problems lies in NP can also be falls in class- P problems.

That means, some of the NP problems can be solved within polynomial time using deterministic algorithms in II stage of non-deterministic algorithms.

- 2) whether $P=NP$? @ $P \neq NP$? **MADHUSUDHAN M.V.**

That means, problems which are known to lie in P are often called as tractable. problems which lie outside of P are often termed as intractable.

Thus, question of $P=NP$ @ $P \neq NP$ is same as that of asking whether there exists problems in NP which are tractable @ not.

Thus, relationship between P & NP is as shown in fig 9.2

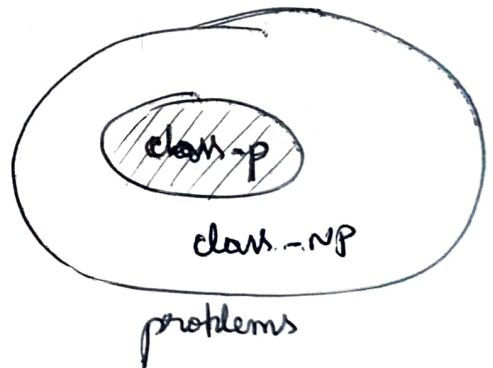


Fig 9.2

We know that, most of the decision problems falls in class- P problems. But, few decision problems are in class- NP . So, we can say

"A decision problem 'D' is said to be NP-complete, if

* it belongs to class- NP

* every problem in NP is polynomially reducible to P . "

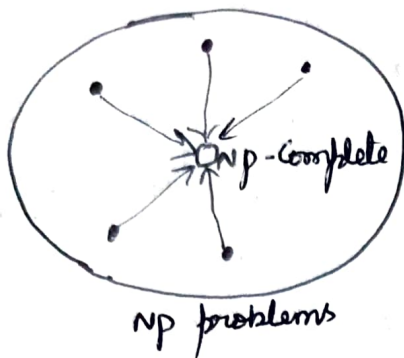


Fig 9.3

That means, if a problem which is in NP & if that can be reduced to a problem which can be solved in polynomial time, then we can say that problem as NP-Complete.

If a decision problem lies in class-P & as well as class-NP, then we can say that $P = NP$. If any decision problem having this relationship of $P = NP$ then that decision problem is called NP-hard.

(OR)

MADHUSUDHAN M.V.
Dept. of CSE

A NP problem such that, if it is in P, then $NP = P$. If a problem has this $NP = P$ property, then it is called as NP-hard problem.